# Estructura del programa archivo app.py

```
from flask import Flask, render_template, request, redirect, url_for, session
import sqlite3
import os

app = Flask(__name__)
app.secret_key = 'clave_secreta_super_segura_tienda'

UPLOAD_FOLDER = 'static/uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

USUARIO_CORRECTO = "Elcallejon"
PASSWORD_CORRECTO = "Lamoda44"
```

- Usamos un flask para poder levantar los archivos en un servidor para que pueda mostrar la página web también para poder mostrar los diseños que se crearon con el lenguaje HTML
- Se uso un Sqlite3 como libreta digital para poder guardar toda la información y para poder guardar los datos sin que se pierda nada al cerrar el programa
- Se uso un Secret Key para poder iniciar sesión de una manera segura
- El upload folder sirve para usar la carpeta static donde se va a guardar las imágenes de los artículos que se suban a la pagina
- La parte de Usuario y contraseña correctos es donde se establece cuáles son las credenciales establecidas para poder ingresar a la página de inventario, si no se ponen las credenciales que están establecidas en el código no se va a permitir ingresar al usuario

```
def init_db():
    conn = sqlite3.connect('inventario.db')
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS productos (
            ...
        )
    ''')
    conn.commit()
    conn.close()
```

- El tener la función de init_db es la que se encarga de poder conectar un archivo llamado inventario.db o crear este si no existe lo que hace que se ejecute una sentencia en SQL que asegura que en la tabla de productos esta lista en columnas como el nombre, la categoría, la talla, el color, el precio y el número que hay en stock y también subir la imagen

```
def ordenamiento_burbuja(lista, criterio='id'):
    ...
```

- Se uso la función de ordenamiento de burbuja que es la que ayuda a hacer la lista de productos que se obtienen en la base de datos, el criterio se utiliza como un diccionario en el índice como el id, el nombre o el precio, esto ayuda a saber de manera exacta que columna debe de comparar

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    ...

@app.route('/logout')
def logout():
    ...
```

- El sistema de autenticación, la función de /login ayuda a manejar la visualización de la página de inicio y también la validación de las credenciales enviadas, si las credenciales son las correctas se valida la sesión y el /logout es el encargado de cerrar la sesión que este activa y esto hace que se elimine la variable de la sesión y esto lo vuelve a dirigir al login

```
@app.route('/')
def index():
    if not session.get('logged_in'):
        return redirect(url_for('login'))
    ...
```

- La vista principal de la página se conforma con el control de acceso que es el encargado de verificar que el usuario haya iniciado una sesión
- La busque y el filtro es el que captura un parámetro de búsqueda que ayuda a filtrar los productos por su nombre, su categoría, el color y su talla que consulta mediante el SQL mientras se usan operadores like
- El procesamiento y la estadística ayuda a pasar los resultados mediante la función de ordenamiento de burbujas, también ayuda a calcular el total de artículos que hay en stock, la variedad de productos, el precio promedio y muestra cual es la prenda más cara y la más barata que lo empaqueta en un diccionario que se llama cálculos
- La parte del renderizado es el encargado de devolver a la plantilla principal que el index.html, este es el encargado de enviarle los productos de una manera ordenada junto con los cálculos estadísticos y los filtros que son aplicados

```python
@app.route('/agregar', methods=['POST'])
def agregar():
    ...

@app.route('/eliminar/<int:id>')
def eliminar(id):
    ...
```

- La gestión de los productos tiene una función de /agregar la cual es la encargada de recibir los datos en el formulario de un nuevo producto juntos con todos los datos y el archivo de su imagen del producto, este es el encargado de validar que los valores numéricos no sean negativos, este guarda la imagen de manera física en la ubicación de la carpeta static y también registra el producto nuevo en la base de datos
- La función de /eliminar/<int:id> es el encargado de buscar el producto por su ID, también hace la función de borrar el archivo de la imagen que está asociado en el disco duro, esto se hace para no dejar que se acumule basura en el servidor y como paso final elimina el registro en la base de datos antes de poder recargar la página principal

```python
if __name__ == '__main__':
    init_db()
    app.run(debug=True)
```

- Para poder inicializar el programa la base es la encargada de llamar al archivo init_db y esto hace que se arranque el servido local desarrollado de flask junto al modo de depuración que esta activado

# Estructura y explicación del archivo index.html

```
<!-- Encabezado con Logo y Texto 3D -->
<header class="d-flex justify-content-between align-items-center mb-5 pb-3 border-bottom border-secondary">
    <div class="d-flex align-items-center gap-3">
        <img src="{{ url_for('static', filename='uploads/logo de la tienda.jpg') }}" alt="Logo" class="header-logo">
        <div>
            <h1 class="title-3d mb-0">El Callejón de la Moda</h1>
            <p class="subtitle-3d mb-0">Sistema de monitoreo e inventario urbano</p>
        </div>
    </div>
    <a href="/logout" class="btn btn-outline-light btn-sm fw-semibold">Cerrar Sesión</a>
</header>
```

- La parte del encabezado y la identidad visual es el encargado que muestra el logotipo de la tienda con las especificaciones y el diseño que se estableció como el tener un borde dorado, el titulo principal con los efectos
- También tiene un control de sesión rápido el cual es el que incorpora un botón con un diseño limpio el cual es el que sirve para cerrar la sesión en la página, este está vinculado de manera directa en la ruta de /logout, esto sirve para salir de la aplicación de una manera segura

```
<div class="row g-3 mb-4">
    <div class="col-md-3">
        <div class="card metric-card p-3">
            <span class="text-muted small text-light">Total en Stock</span>
            <h4 class="fw-bold text-white mb-0">{{ calculos.total_prendas }}</h4>
        </div>
    </div>
    ...
</div>
```

- Este es el panel que contiene los indicadores que son claves en el programa, este hace la presentación en tiempo real como las métricas estadísticas que son calculadas desde el servidor, también están organizadas en tarjetas con los bordes resaltados
- La parte de total del stock realiza una suma de manera global en las todas las unidades que están disponibles
- La variedad de los artículos es la cantidad total del registro que son diferentes en la base de datos
- El precio promedio hace que se muestre el costo medio de las prendas y hace que se expresen en quetzales
- Los extremos de los precios son los que muestran de una forma dinámica donde muestra la prenda más cara y la prenda más barata que son registradas

```
<form action="/agregar" method="POST" enctype="multipart/form-data">
    ...
</form>
```

- El formulario de los registros es el encargado de la creación de los artículos, este habilita la entrada de los datos que son estructurados para poder dar de alta a las prendas nuevas con los atributos específicos
- También hay una sección que tiene unos campos de texto que son obligatorios para el apartado de nombre, categoría y el valor
- Hay una sección de un selector que se despliega el cual esta divido entra las tallas de ropa que van desde la talla XS hasta la talla XXL y también el número de talla del calzado el cual va desde el número de talla 35 hasta el número de talla 45
- Hay controles números que sirven para poder establecer el precio y el stock y también hay un selector para los archivos multimedia que es adaptado de manera exclusiva para poder tener una imagen del producto

```
<form method="GET" action="/" class="row g-2 align-items-center">
    <input type="text" name="busqueda" placeholder="Buscar por producto, talla o cat
    <select name="orden">
        <option value="id">Ordenar por: ID</option>
        <option value="nombre">Ordenar por: Nombre (A-Z)</option>
        <option value="precio">Ordenar por: Precio (Burbuja)</option>
        <option value="stock">Ordenar por: Cantidad (Burbuja)</option>
    </select>
</form>
```

- Esta sección es un selector de ordenamiento y también un buscador, esto permite que se pueda filtrar de manera dinámica los registros mediante un método de consulta de texto y así poder reorganizar la tabla seleccionando los criterios avanzados y también incluyendo el algoritmo de la parte de ordenamiento de burbujas para los precios y los stocks

```
<table class="table table-hover align-middle mb-0">
    <tbody>
        {% for p in productos %}
        <tr>
            ...
        </tr>
        {% endfor %}
    </tbody>
</table>
```

- Este apartado es una tabla de inventario dinámica, esta sobre la lista de los productos que son procesados durante el servidor mediante una plantilla Jinja2 que ayuda a demostrar cada articulo entre las filas interactivas
- La visualización de la multimedia ayuda a mostrar una miniatura de la imagen sobre el producto que se va agregando a la página web
- El etiquetado inteligente es el encargado de resaltar las tallas junto a las insignias personalizadas y también evalúa el nivel del inventario lo que hace también que se muestre una alerta roja, esta alerta indica que el stock es bajo o puede que se muestre en un color verde el cual indica que el stock es lo contrario, es decir que es un stock alto
- La acción de borrado tiene incorporado un enlace de eliminación que está protegido con una ventada de confirmación ejecutado con JavaScript el cual se realiza antes de enviar la petición al servidor

# Estructura y explicación del archivo login.html

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Acceso - El Callejón de la Moda</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
```

- Esta parte es la encargada de cargar los estilos y el diseño visual, tiene agregado un Bootstrap vinculada en una red de distribución de contenido para poder estructurar la interfaz de una manera adaptable y limpia
- Este define los estilos que fueron personalizados ubicado en el bloque style para poder configurar el fondo el cual fue personalizado con un difuminado y también el diseño de varias partes como el diseño de la tarjeta de acceso, también los bordes y los botones interactivos

```
<body class="d-flex align-items-center justify-content-center">
```

- Esta es la encargada de realizar un centrado de manera automática, esta utiliza la clase de bootstrap para poder centrar de manera perfecta todo el contenido como la tarjeta de inicio de una sesión, tanto de manera una forma vertical, así como también horizontal en la pantalla del dispositivo

```
<div class="card-body text-center">
    <img src="{{ url_for('static', filename='uploads/logo de la tienda.jpg') }}" alt="Logo Tienda" class="login-logo mb-3">

    <h3 class="fw-bold text-white mb-1">EL CALLEJÓN</h3>
    <p class="small mb-4 fw-semibold tracking-widest text-uppercase" style="color: #d4af37 !important;">de la moda</p>
```

- Este apartado es el encargado de la identidad visual del formulario el cual contiene la tarjeta principal del login el cual esta personalizado con un sombreado
- Este se carga de una manera dinámica al logotipo oficial que se seleccionó para la tienda desde la ubicación de una carpeta de archivos estáticos para el servidor mediante la función del flask url for
- También se muestra el titulo y el subtítulo con los tonos que fueron seleccionados para representar la marca

```
{% if error %}
    <div class="alert alert-danger text-center p-2 mb-3 small bg-danger text-white border-0">{{ error }}</div>
{% endif %}
```

- Este apartado es un control de alertas dinámicas que se ejecuta mediante una condición Jinja2 el cual evalúa si Python pudo enviar un mensaje de error en caso de que el usuario no haya ingresado las credenciales correctas, si en cierto caso las credenciales no hayan sido las correctas se muestra una alerta en la pantalla que le notifica al usuario sin que sea necesario refrescar la pagina

```
<form action="/login" method="POST" class="text-start">
    <div class="mb-3">
        <label class="form-label text-light small">Usuario</label>
        <input type="text" class="form-control bg-dark text-white border-secondary" name="usuario" required placeholder="admin">
    </div>
    <div class="mb-3">
        <label class="form-label text-light small">Contraseña</label>
        <input type="password" class="form-control bg-dark text-white border-secondary" name="password" required placeholder="••••">
    </div>
    <button type="submit" class="btn btn-gold w-100 mt-2 py-2">Ingresar al Sistema</button>
</form>
```

- Este apartado es el encargado del formulario de autenticación el cual es configurado con el método post el cual es el encargado de enviar los datos de una manera segura buscando la ruta /login mediante el servidor backend
- Este también incluye los campos de entrada de texto que son obligatorios para poder computarár el apartado de usuario y de contraseña
- El poder incorporar un botón que realice la acción de envió el cual abarca todo el ancho del contenedor para poder procesar un acceso al sistema

# ¿Porque se creó el programa con lenguaje HTML y lenguaje Python?

Este proyecto se creó combinando dos lenguajes de programación los cuales fueron Python y también el lenguaje HTML por que el archivo creado con lenguaje Python va a funcionar como el motor que va a procesar los datos, también es el que gestiona la base de datos y también hace que las sesiones sean seguras y los archivos HTML es la que realiza toda la interfaz gráfica y visual para que sea funcional para el usuario final

El jinja2 funciona como el motor de plantilla que utiliza el flask y Python, esto sirve para conectar el servidos con la página web escrita en HTML, esto sirvió para insertar algunos datos dinámicos de Python de manera directa con el diseño visual de la página, el implementar el Jinja2 ayuda a que cada vez que se agrega un nuevo producto se pueda recorrer en la lista y así se pueda pintar una nueva fila, también ayuda a que en el stock muestre una alerta roja si el número del stock es bajo o verde si es que hay suficientes unidades

Los archivos que se escribieron en el lenguaje de HTML sirvieron para crear la interfaz visual y la presentación de la aplicación, con estos archivos se pudo crear las ventanas de bienvenida y el de la seguridad de la tienda, así como también ayudo a mostrar la interfaz donde aparece el logotipo que se

estableció para la tienda, esto ayudo a traducir los códigos y datos de Python a convertirlos en una experiencia visual que sea atractivo

# ¿Porque se utilizó un servidor en render?

Se tuvo que utilizar un servidor aparte como render porque el programa web es de flask y se diseñó para poder funcionar en un servidor real, se utilizó render porque en Gib Hub el apartado de page donde normal mente brinda el link para acceder a la página no acepta archivos que contengan un entorno flask, solo acepta archivos con lenguaje HTML, este no acepta archivos Python ni base de datos SQLite

# Manual de usuario

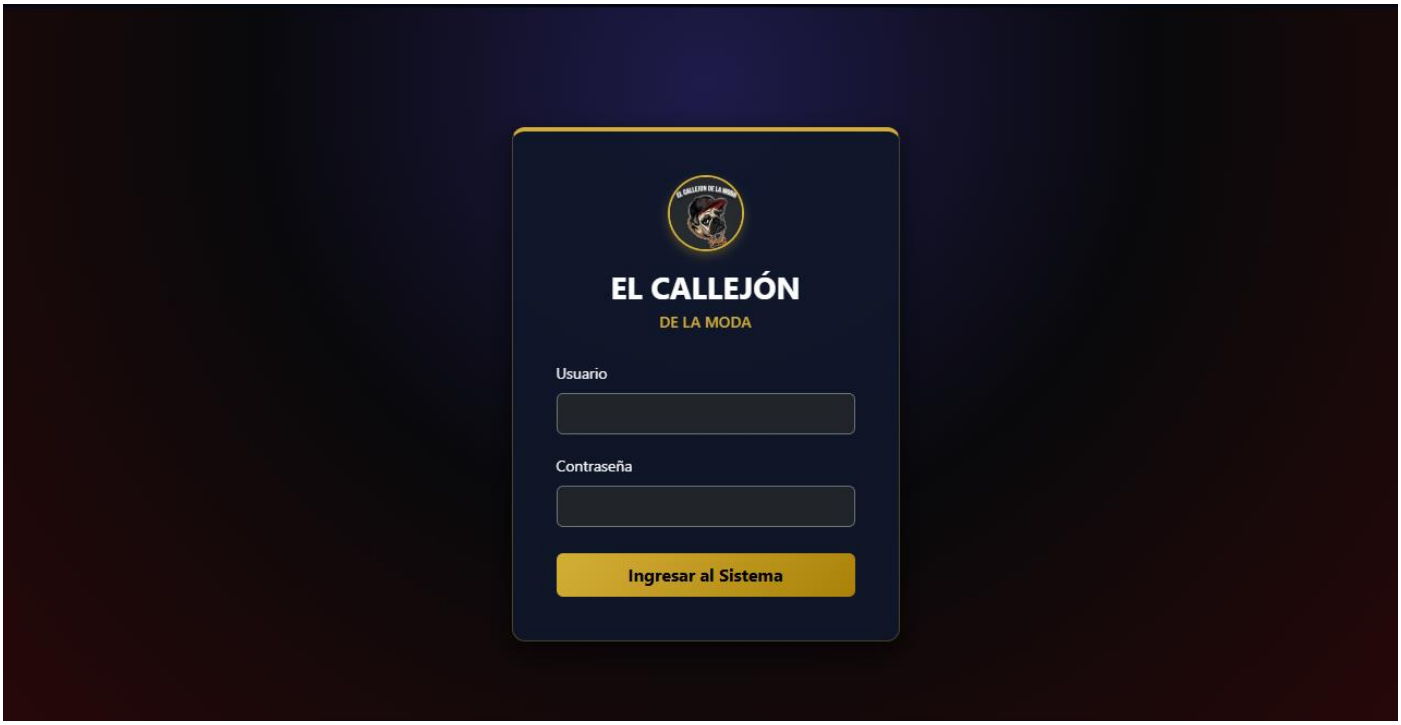
Abrir el siguiente enlace

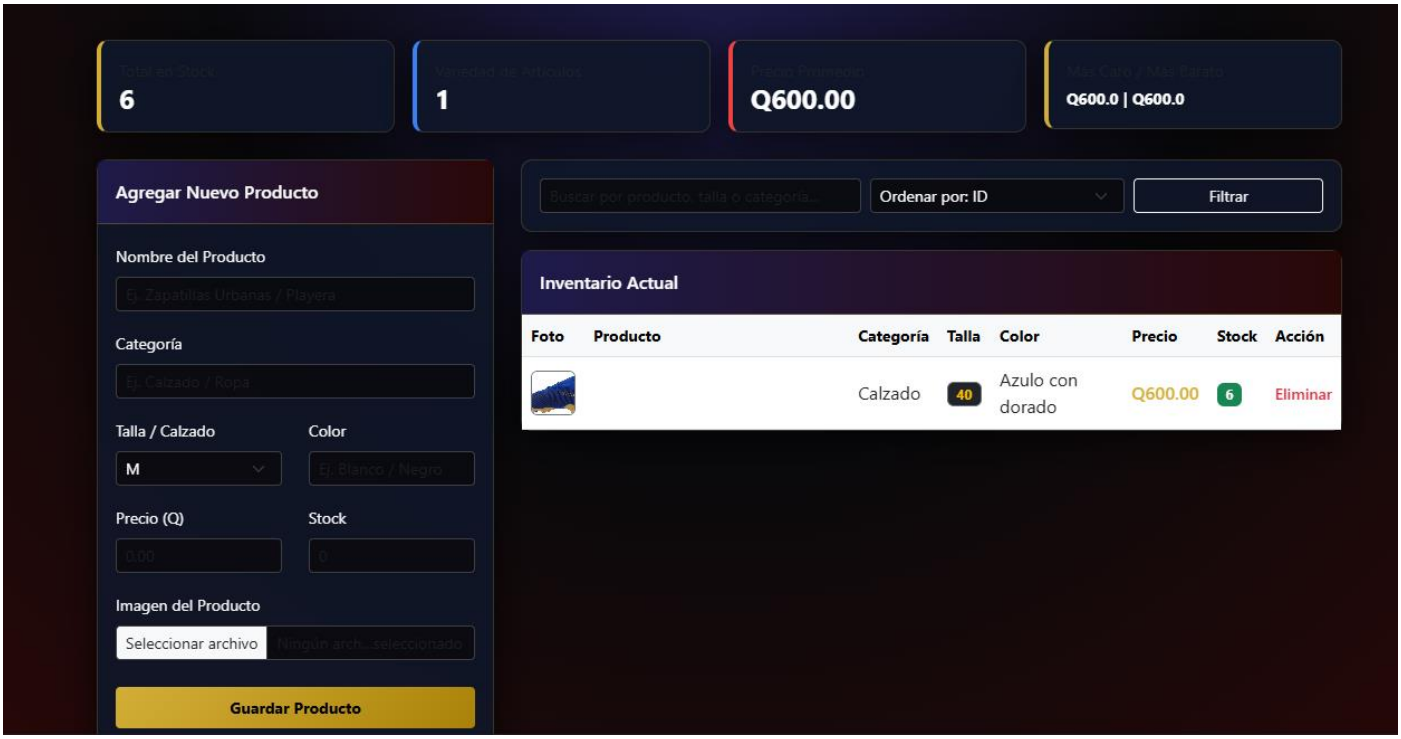
https://inventario-el-callejon-de-la-moda.onrender.com/

Usar las siguientes credenciales

Usuario: Elcallejon

Contraseña : Lamoda44






En este apartado puede subir la imagen sobre el artículo que desea publicar y también escribir las características y seleccionar la talla o el número de calzado